OS Assignment 4

Name: Mathew K Alexander
SRN: PES2UG24CS270

Github link: https://github.com/mrwhats-it/PES2UG24CS270-pes-vcs

Phase 1:

1A

```
[alexa@arch PES2UG24CS270-pes-vcs]$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
[alexa@arch PES2UG24CS270-pes-vcs]$
```

1B

```
[alexa@arch PES2UG24CS270-pes-vcs]$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
[alexa@arch PES2UG24CS270-pes-vcs]$
```

Phase 2:

2A

```
[alexa@arch PES2UG24CS270-pes-vcs]$ make test_tree
./test_tree
make: 'test_tree' is up to date.
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
[alexa@arch PES2UG24CS270-pes-vcs]$
```

2B

```
[alexa@arch PES2UG24CS270-pes-vcs]$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f62 2031 3600 4865 6c6c 6f2c 2050  blob 16.Hello, P
00000010: 4553 2d56 4353 210a                      ES-VCS!.
```

Phase 3:

3A

```
[alexa@arch PES2UG24CS270-pes-vcs]$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
make: 'pes' is up to date.
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree
  untracked:  pes.h
  untracked:  tree.c
  untracked:  commit.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  test_objects.c
  untracked:  index.h
  untracked:  pes.c
  untracked:  README.md
  untracked:  Makefile
  untracked:  object.c
  untracked:  commit.h
  untracked:  tree.h
  untracked:  .codex
  untracked:  index.c
  untracked:  test_objects
  untracked:  .gitignore
```

3B

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776746948 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776746948 6 file2.txt
```

Phase 4

4A

```
[alexa@arch PES2UG24CS270-pes-vcs]$ ./pes log
commit 6c0d10cf5cce41d36637a76ffad1864161062e3e4e53dd92108fec05592941e4
Author: PES User <pes@localhost>
Date:   1776747122

    Add farewell

commit b00ec99b808c094f26c9d18334fe68633cc8470619f9c9f69110060db86eb8c7
Author: PES User <pes@localhost>
Date:   1776747122

    Add world

commit a46ae051023d810e52d5a96021a44a31562cb3e365bce45afa45ba76ce0cb5cb
Author: PES User <pes@localhost>
Date:   1776747122

    Initial commit

commit 426cf2eee7a2c15096d0c36de9375c563f0d50e723e649415172a7b371c20554
Author: Mathew K Alexander <PES2UG24CS270>
Date:   1776747089

    Add farewell

commit 804c9fcf420462923db57ca4f80d49c97dcfd72463e7e230cd2f875f4758de98
Author: Mathew K Alexander <PES2UG24CS270>
Date:   1776747089

    Add world

commit ddaa3b4903f3708bd4a6b7d38280ff076a821b137f4c8d8c6c9ddcf1d344a16b
Author: Mathew K Alexander <PES2UG24CS270>
Date:   1776747089

    Initial commit

[alexa@arch PES2UG24CS270-pes-vcs]$
```

4B

```
[alexa@arch PES2UG24CS270-pes-vcs]$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/3b/c19934c2ccc37b9582dd42105ca4a1df8e197f51c99a3ef71628231437fdbf
.pes/objects/42/6cf2eee7a2c15096d0c36de9375c563f0d50e723e649415172a7b371c20554
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6c/0d10cf5cce41d36637a76ffad1864161062e3e4e53dd92108fec05592941e4
.pes/objects/80/4c9fcf420462923db57ca4f80d49c97dcfd72463e7e230cd2f875f4758de98
.pes/objects/a4/6ae051023d810e52d5a96021a44a31562cb3e365bce45afa45ba76ce0cb5cb
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/b0/0ec99b808c094f26c9d18334fe68633cc8470619f9c9f69110060db86eb8c7
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/dd/aa3b4903f3708bd4a6b7d38280ff076a821b137f4c8d8c6c9ddcf1d344a16b
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4C

```
[alexa@arch PES2UG24CS270-pes-vcs]$ cat .pes/refs/heads/main
6c0d10cf5cce41d36637a76ffad1864161062e3e4e53dd92108fec05592941e4
[alexa@arch PES2UG24CS270-pes-vcs]$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5 and 6

Q5.1

pes checkout <branch> needs to:

1. Resolve target commit: read .pes/refs/heads/<branch>.
2. Update HEAD to ref: refs/heads/<branch> (or direct hash for detached mode).
3. Read target commit object, then its root tree.
4. Reconstruct working directory to match that tree:

- Write tracked files from blobs
- Create/remove directories
- Delete tracked files not present in target tree

5. Rebuild/update index to match checked-out tree metadata.

Why complex:

- Must avoid clobbering uncommitted work.
- Needs recursive tree traversal and path-level conflict handling.
- Must be atomic enough that partial failures don't leave mixed states.

Q5.2
Detect dirty-checkout conflict using index + object store:

1. For each tracked path that differs between current branch tree and target branch tree:

- Compare working file against index entry metadata (mtime, size) as fast filter.
- If metadata differs, hash current working file as blob candidate and compare with indexed blob hash.

2. If working file content != indexed blob and path also changes across branches, block checkout.
3. Also block when untracked working files would be overwritten by target tree paths.

Core idea: "local modification" = working content diverges from index; "checkout danger" = same path diverges between current/target trees.

Q5.3
Detached HEAD means HEAD points directly to a commit hash, not a branch ref. New commits
are created normally, but no branch name advances, so those commits can become unreachable
later.

Recovery:

1. Find commit hash (from log, reflog-style history, or printed commit IDs).
2. Create/move a branch ref to it: e.g. write hash to .pes/refs/heads/recovered.
3. Set HEAD to ref: refs/heads/recovered to continue safely.

Q6.1
GC algorithm:

1. Build root set from all refs (.pes/refs/heads/*, tags if any, detached HEAD hash if applicable).
2. Traverse graph (DFS/BFS):

- Commit -> parent commit(s)
- Commit -> root tree
- Tree -> child trees + blobs

3. Store visited hashes in a hash set (unordered_set-style) for O(1) membership.
4. Scan .pes/objects/**; delete any object hash not in reachable set.

For 100,000 commits and 50 branches, visited commits are usually near ~100,000 total (branches overlap heavily), not 5,000,000. Total visited objects = commits + trees + blobs reachable from those commits; typically many more than commits, but still linear in reachable graph size.

Q6.2
Why dangerous concurrently:

- Commit process may write new objects and later update branch ref.
- If GC runs between those steps, new objects may appear unreachable (no ref yet) and get deleted.
- Then commit finishes updating ref to hashes that GC already removed -> broken history/reference.

Example race:

1. Commit writes tree/blob/commit objects.
2. Before ref update, GC marks reachability from current refs only.
3. GC deletes those newly written objects as unreachable.
4. Commit updates refs/heads/main to deleted commit hash.

How real Git avoids this:

- Uses object liveness protections (reflogs, grace periods, mtimes/prune windows).
- Uses locking/coordination and conservative pruning.
- Typically prunes only sufficiently old unreachable objects, reducing race risk drastically.